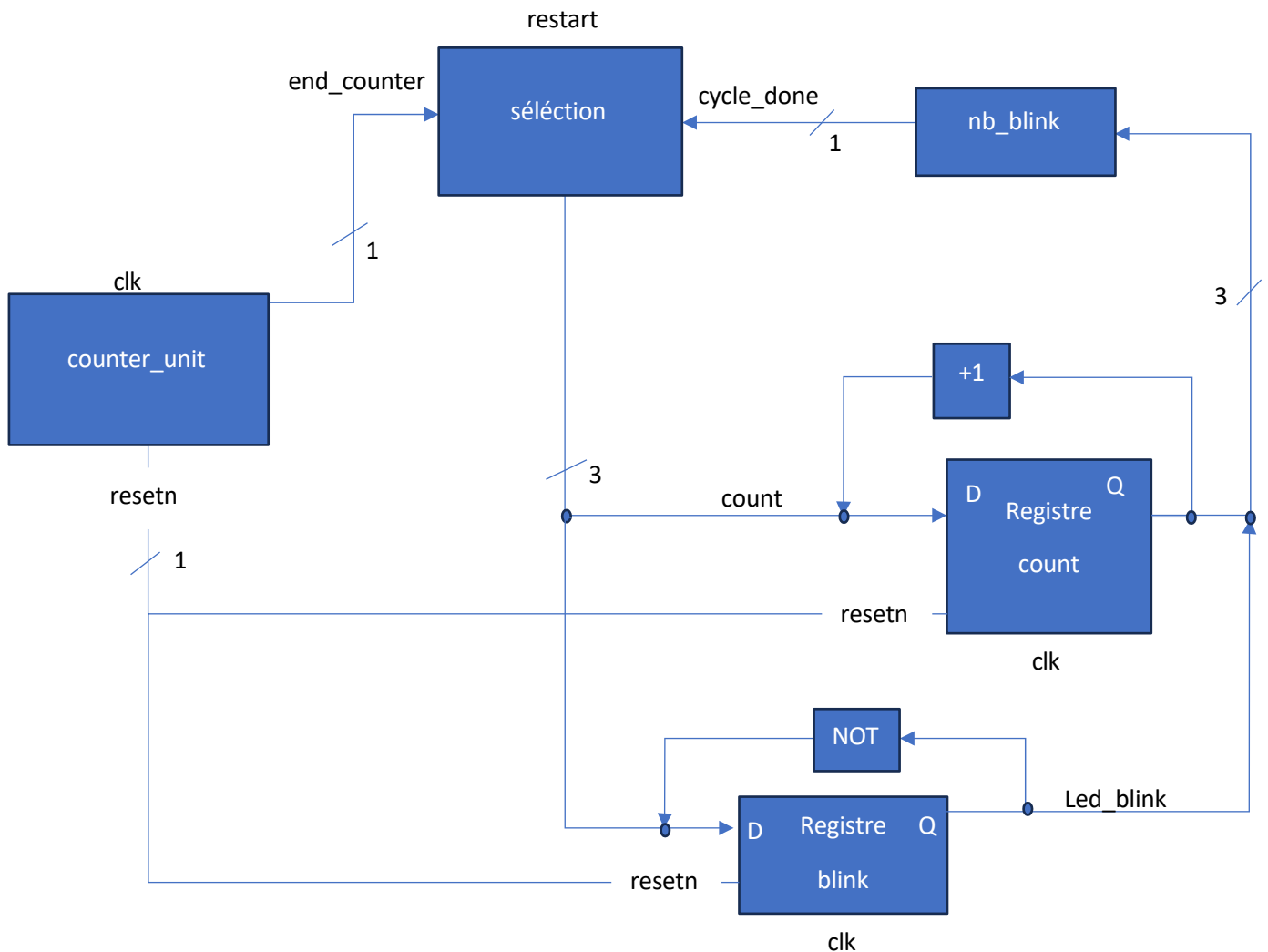


TP3_LED_RGB

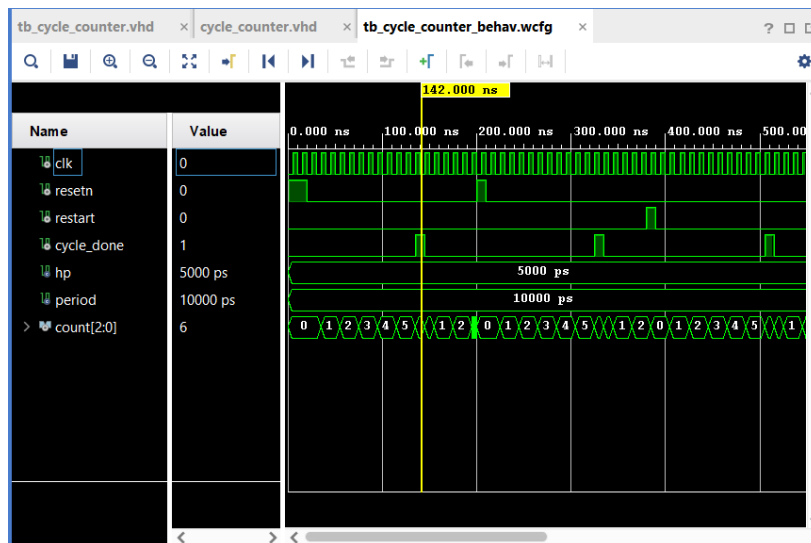
1. Dans un fichier .vhd, créez un module Counter_unit à partir du compteur du TP1. Le module prendra en entrée un signal d'horloge et de resetn, et donnera en sortie le signal end_counter. Utilisez un paramètre generic() pour définir le nombre de coup d'horloge à compter.
2. En schéma RTL, créez un compteur du signal end_counter. Ce compteur doit permettre de déterminer le nombre de cycles allumé/éteint qui ont été effectués par la LED. Le compteur doit pouvoir être remis à 0, maintenir sa valeur actuelle ou s'incrémenter.



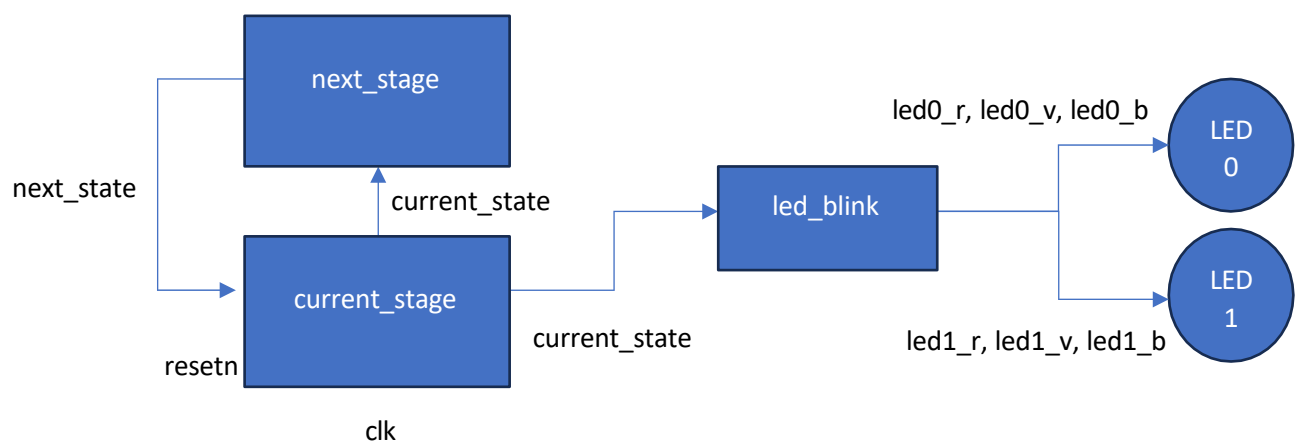
3. Ecrivez un code VHDL décrivant ce compteur de cycle, vous utiliserez le module Counter_unit.

4. Tester votre architecture avec un testbench.

La simulation du testbench est correcte et respecte bien les 6 scénarios de test que j'ai implémenté !



5. Créez en RTL une machine à états (FSM) permettant de faire clignoter une LED RGB en rouge puis bleu et enfin en vert avant de recommencer le cycle (rouge, bleu, vert, ...). Dans chaque état la LED devra clignoter 3 fois. De plus, si le bouton restart est appuyé, on retourne dans l'état initial quel que soit l'état dans lequel on se situe. L'état initial est l'état dans lequel on se situe au démarrage, on passe à l'état rouge après 3 clignotements de la LED en blanc (rouge, vert et bleu actifs en même temps).



6. Listez les signaux d'entrée, de sortie et les signaux internes de votre architecture.

Entrées :

clk : l'horloge
resetn : remet le système à son état initial
restart : bouton utilisateur, permet un retour à l'état initial

Sorties :

Instructions pour piloter les leds 0 et 1 (rouge, vert bleu)

led0_r
led0_v
led0_b
led1_r
led1_v
led1_b

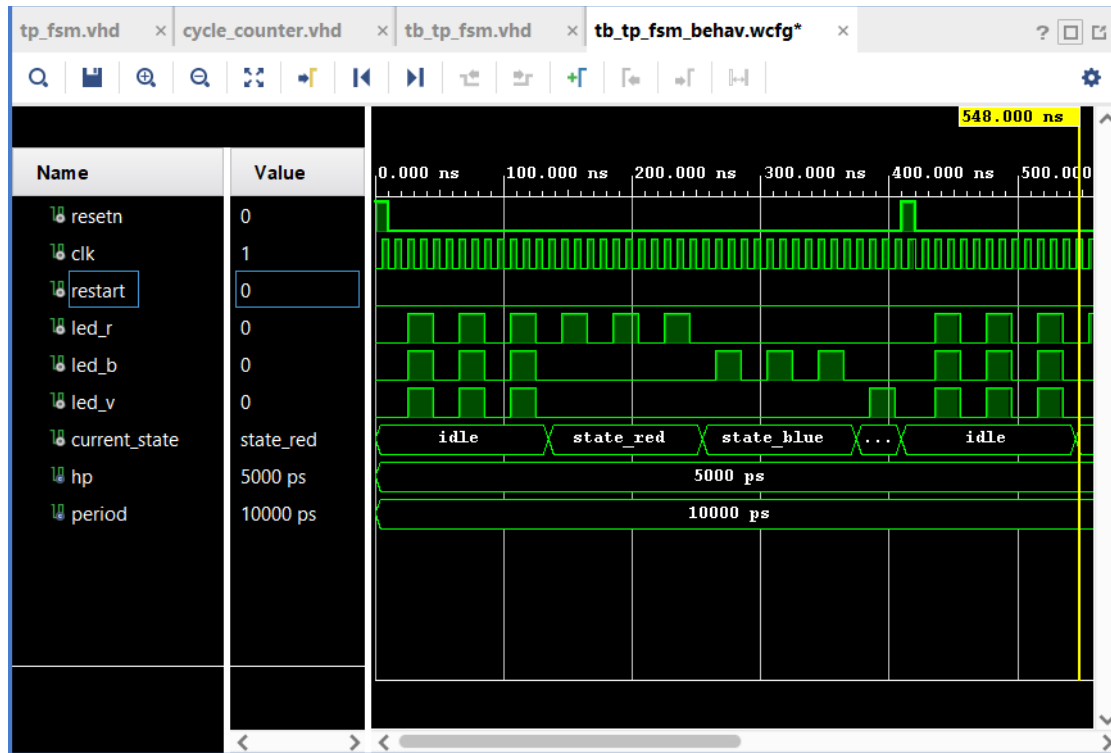
Signaux internes :

current_state : signal possédant l'état actuel de la machine à état
next_state : signal possédant le prochain état de la machine à état
cycle_done : signal indiquant qu'un cycle de clignotement est terminer

7. Ajoutez à votre code VHDL les éléments que vous venez de créer.

8. Ecrivez un testbench pour tester votre architecture. Vérifiez à la simulation que vous obtenez le résultat attendu.

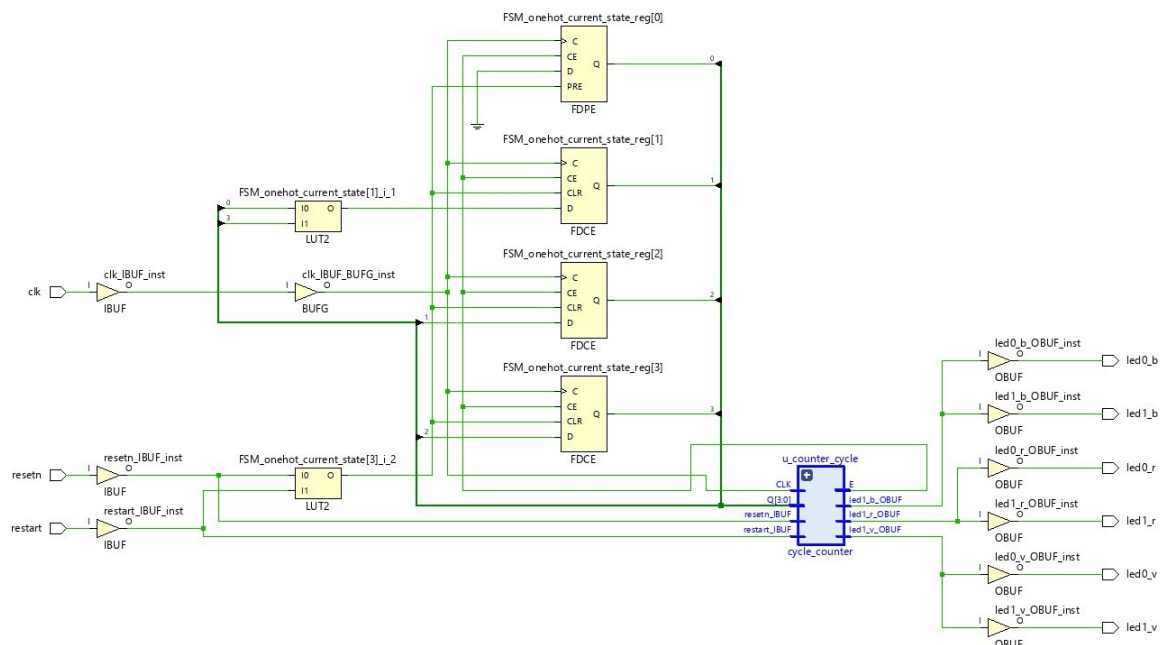
Avec cette simulation on voit bien que les signaux led sont à 1 au bon moment, on peut observer 3 clignotements en blanc après le reset ainsi que 3 rouges, 3 bleus puis 1 vert avant le test du bouton restart, puis à nouveau 3 clignotements blanc.



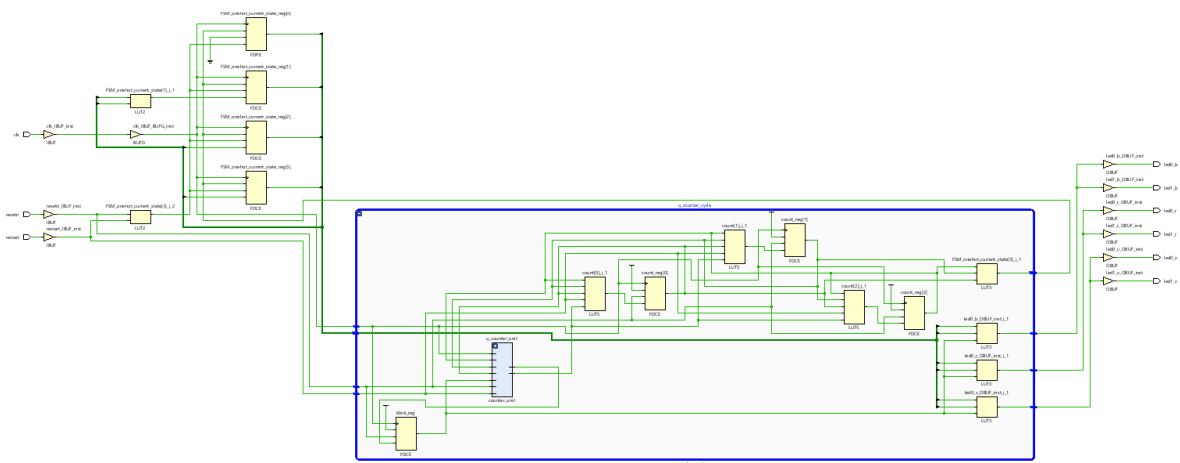
9. Exécutez la synthèse et relevez les ressources utilisées (y compris la FSM). Sur la schématique, identifiez où se situe votre compteur de cycle.

Après avoir exécuté la synthèse et ouvert la schématique, on se rend compte que nos modules sources sont imbriqués les uns dans les autres : counter_unit est dans counter_cycle qui est lui-même dans tp_fsm.

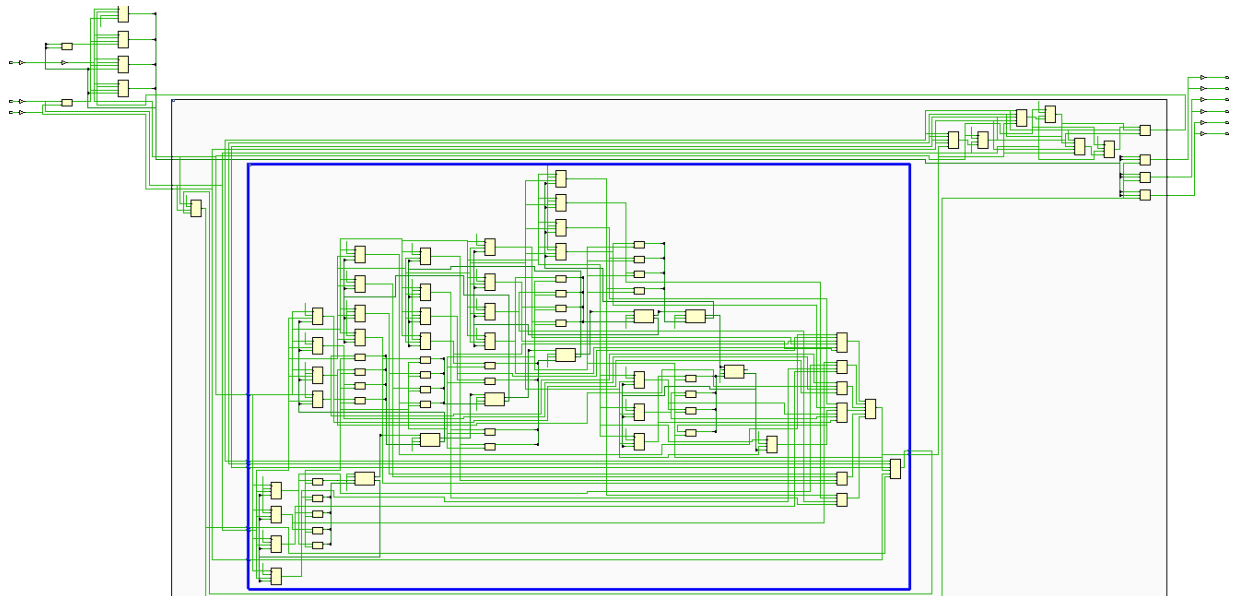
On peut voir la machine à état avant le module cycle_counter en bleu ainsi que les 3 entrées (clk, reset, et restart) et les 6 sorties (voir. Q10), (led0_r, led0_b, led0_v, led1_r, led1_b et led1_v).



Le module cycle_counter à l'intérieur du tp_fsm



Et le module counter_unit dans le cycle_counter.



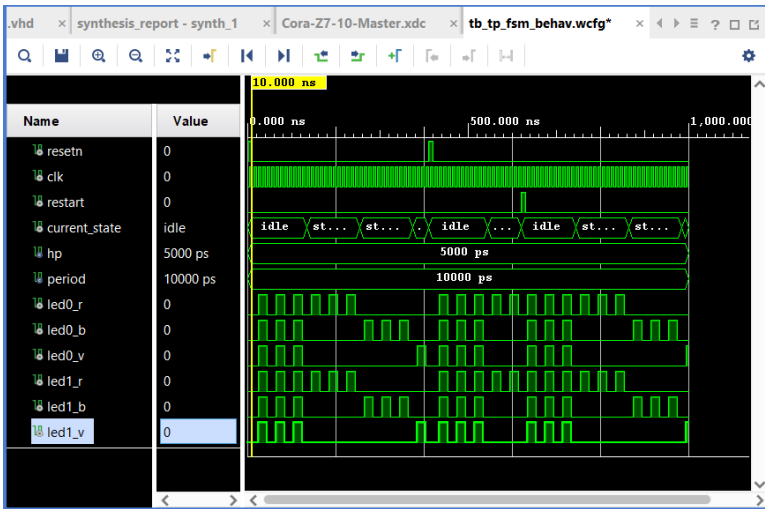
On peut également voir que l'on a bien 3 buffers d'input pour les 3 entrées et 6 buffers d'output pour les commandes leds.

Report Cell Usage:

+-----+-----+-----+		
	Cell	Count
+-----+-----+-----+		
1	BUFG	1
2	CARRY4	7
3	LUT2	31
4	LUT3	4
5	LUT4	4
6	LUT5	3
7	LUT6	4
8	FDCE	35
9	FDPE	1
10	IBUF	3
11	OBUF	6
+-----+-----+-----+		

10. Modifiez le fichier de contraintes pour connecter vos entrées / sorties du système avec les broches de la carte. Réglez l’horloge pour que sa fréquence soit à 100MHz.

Quand j’ai décommenté les lignes je me suis rendu compte qu’on ne pouvait pas piloter les 3 couleurs de 2 leds avec seulement 3 signaux qu’on dupliquerait. J’ai donc modifié mon code pour avoir 6 signaux en sorties et j’ai également ajusté mon test bench afin qu’il tests les 2 leds (j’ai également apporté quelques modifications dans mes réponses à certaines questions) :



11. Lancez l’implémentation puis étudiez le rapport de timing (vérifiez les violations de set up et de hold et identifiez le chemin critique)

On constate que le WNS et le WHS sont positif, de plus on a l’information que « all user specified timing constraints are met » donc tout est en norme.

Design Timing Summary			
Setup	Hold	Pulse Width	
Worst Negative Slack (WNS): 5,088 ns	Worst Hold Slack (WHS): 0,206 ns	Worst Pulse Width Slack (WPWS): 3,500 ns	
Total Negative Slack (TNS): 0,000 ns	Total Hold Slack (THS): 0,000 ns	Total Pulse Width Negative Slack (TPWS): 0,000 ns	
Number of Failing Endpoints: 0	Number of Failing Endpoints: 0	Number of Failing Endpoints: 0	
Total Number of Endpoints: 39	Total Number of Endpoints: 39	Total Number of Endpoints: 37	
All user specified timing constraints are met.			

En sélectionnant le path 11, je constate que les délais sont cohérent, le chemin est correctement exécuté et passe bien par les bon modules.

The screenshot displays the Xilinx Vivado interface. The top pane shows the Netlist with a tree view of components including `blink_reg` (FDC), `count[0]_1` (LUT5), `count[1]_1` (LUT5), `count[2]_1` (LUT5), and `count_reg[0]` (FDC). The middle pane shows the Path Properties for Path 11, with the Summary tab selected. The bottom pane shows the Timing Summary for Path 11, which is highlighted in the Intra-Clock Paths section.

Name	Slack	Levels	High Fanout	From	To	Total Delay	Logic Delay	Net Delay	Requirement	Source Clock
Path 11	0.206	1	5	u_counter_cy_unt_reg[0]/C	u_counter_cy_blink_reg/D	0.297	0.227	0.070	0.0	sys_clk_pin
Path 12	0.228	1	4	FSM_onehot_c_te_reg[0]/C	FSM_onehot_c_te_reg[1]/D	0.319	0.227	0.092	0.0	sys_clk_pin
Path 13	0.254	0	2	FSM_onehot_c_te_reg[2]/C	FSM_onehot_c_te_reg[3]/D	0.324	0.141	0.183	0.0	sys_clk_pin
Path 14	0.308	1	5	u_counter_cy_unt_reg[1]/C	u_counter_cy_unt_reg[2]/D	0.415	0.184	0.231	0.0	sys_clk_pin
Path 15	0.313	2	2	u_counter_cy_unt_reg[3]/C	u_counter_cy_er_reg[3]/D	0.418	0.249	0.169	0.0	sys_clk_pin
Path 16	0.313	2	2	u_counter_cy_er_reg[11]/C	u_counter_cy_er_reg[11]/D	0.418	0.249	0.169	0.0	sys_clk_pin
Path 17	0.313	2	2	u_counter_cy_er_req[27]/C	u_counter_cy_er_req[27]/D	0.418	0.249	0.169	0.0	sys_clk_pin

Timing Summary - impl_1 (saved)

12. Générez le bitstream pour vérifier le système sur carte.

Le bitstream a été généré avec succès, le test sur carte marche parfaitement.

Les 2 leds clignotent en même temps 3 fois de chaque Scouleur et quand le bouton restart ou reset est appuyé, elles clignotent 3 fois en blanc avant de reprendre le cycle. (vidéo de démonstration)